ISM2411 — Lab Week 08

Your First GitHub Submission — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 08 · Unit 2 · Control Flow & Structure

Contents

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 08 Lab — Your First GitHub Submission
Unit	Unit 2 · Control Flow & Structure
Class length	75 minutes
Format	Live terminal/browser code-along — no new Python syntax today; the "code" is Git commands and Markdown
Prerequisites	Module 02 terminal fluency; a `functions.py` file from Module 07 to add to the new repo
Student-facing lab page	markumreed.github.io/ism2411 — week08_lab
Exercises covered	Exercises 1–6 (required) + Stretch 1/2 (as time allows)
Submission	The URL to the student's GitHub repo, submitted to Canvas

This is the second-highest-variance lab of the semester to run, after Module 02, for the same underlying reason: real accounts, real network access, real external services (github.com) are now on the critical path, not just each student's local machine. Authentication issues, two-factor setup friction, and the first-ever `git push` failing for an unexpected reason are all normal here — budget real slack. The actual Git concepts (stage, commit, push, log) are genuinely simple; the friction is almost entirely in the tooling and account setup, not the ideas.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Create a GitHub repository, clone it locally, and explain the relationship between the local folder and the remote repo on github.com.
2. Explain what a `.gitignore` file does and why certain files (compiled bytecode, OS metadata, editor settings) should never be tracked.
3. Execute the stage → commit → push cycle (`git add`, `git commit -m "..."`, `git push`) and describe what each of the three steps actually does.
4. Write a clear, descriptive commit message for a single, coherent change — not a vague "update stuff" message covering several unrelated edits.
5. Read `git log --oneline` output and connect it to what's visible on github.com.

3 Before Class — Setup Checklist

- ☐ If any students don't yet have a GitHub account, this needs to happen *before* class if at all possible (Exercise 1's account creation, especially with two-factor authentication setup, can eat 10+ minutes per student if left for class time) — send a reminder before this session.
- ☐ Confirm `git` is installed and configured on your own demo machine (`git config --global user.name` / `user.email` set) before class, and check a few students' machines during a settling-in period, since a missing global Git identity produces a confusing error on a student's very first commit.
- ☐ Decide your authentication strategy for `git push` in advance: GitHub no longer accepts a plain password over HTTPS — students will need either a **personal access token** (used in place of a password) or SSH keys set up. Pick one method to standardize on for the whole room and know its exact steps before class, rather than improvising live; this is the single most common blocker in this entire lab.
- ☐ Have your own example repo (or a scratch one built live) ready to demo the full exercise sequence once, end to end, before students start their own.

4 Materials Needed

- A GitHub account (github.com) — created before class if possible
- `git` installed locally (confirmed in Module 02's precourse setup)
- A terminal and text editor
- `functions.py` from Module 07 (needed for Exercise 4)

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:05	Welcome: why version control, and why this workflow repeats all semester	5
0:05–0:15	Exercise 1 — Create the account	10
0:15–0:25	Exercise 2 — New repo, clone locally	10
0:25–0:35	Exercise 3 — Add a `.gitignore`	10
0:35–0:43	Exercise 4 — Add Module 7 work	8
0:43–0:51	Exercise 5 — Write a README	8

Time	Segment	Minutes
0:51–0:59	Exercise 6 — Three commits, `git log`	8
0:59–1:07	Stretch 1 — Back-fill earlier modules	8
1:07–1:15	Stretch 2 preview + wrap-up, reflection, submission checklist	8

This plan assumes accounts already exist per the setup checklist above; if account creation genuinely has to happen live for several students, expect Exercise 1 to run long and treat Stretch 1/2 as the first things to compress or cut, not Exercises 2–6, which establish the actual workflow every remaining lab this semester depends on.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:05)

Say to the class:

> *"Zero new Python today. Instead, you're learning the tool that every remaining assignment this semester — and very likely every software job you'll ever have — uses to save and submit work: Git and GitHub. Git is the tool that tracks changes to your files over time, on your own machine. GitHub is a website that hosts a copy of that history, so you (and eventually collaborators, and me, for grading) can see it. Today you set up the exact workflow you'll repeat, with almost no variation, for the rest of the semester: stage a change, commit it with a message explaining what you did, push it to GitHub."*

Do: Write the three-step cycle on the board, large, and leave it visible all class: `git add` → `git commit -m "..."` → `git push`. Every exercise from here returns to this cycle.

6.2 Exercise 1 — Create the Account (0:05–0:15, 10 min)

Teaching goal: A professional, durable GitHub account and username — this is infrastructure students will reuse for the rest of their careers, not just this course.

Say to the class:

> *"If you already have a GitHub account from a previous class or project, you can use it — skip ahead. If not: create one at github.com, using your USF email. Pick your username carefully — this is going to show up on your portfolio, potentially to future employers. `jsmith2024coursework` is a worse choice than `jane-smith-dev` or just your real name if it's available."*

Do: Walk through account creation once on the projector, narrating each step, then let the room work independently while you circulate.

Line-by-line explanation of what each setup step actually does:

- **Username choice** — this becomes part of every repo URL you create (`github.com/yourusername/reponam` and cannot be trivially changed later without breaking old links — worth the extra thirty seconds of thought this deserves.
- **Enabling two-factor authentication (2FA)** — GitHub increasingly requires this for anything beyond read-only access; setting it up now, calmly, in class, avoids being blocked later at a less convenient moment (like the night an assignment is due). Recommend an authenticator app (not SMS) if your students' phones support it — more reliable and it's what GitHub itself recommends.

Common student mistakes to watch for:

- Using a personal, informal email instead of checking whether their USF email offers any GitHub education benefits (free access to certain paid features, relevant for students who go on to use GitHub Pro/Copilot benefits) — not required for this course, but worth a one-sentence mention.
- Setting 2FA with SMS to a phone number they're likely to change soon (e.g., a temporary or family member's number) — a good moment to recommend an authenticator app instead, which isn't tied to a phone number.

Check for understanding: Have each student navigate to their own profile page (`github.com/yourusername`) and confirm it loads with their chosen username visible — a simple visual verification, matching the lab page's own "VERIFICATION" note.

6.3 Exercise 2 — New Repo (0:15–0:25, 10 min)

Teaching goal: Create a repository on GitHub, then `clone` it — bring a working copy down to the local machine — and understand that these are two distinct, connected things: a remote copy (on GitHub's servers) and a local copy (on the student's own machine).

Say to the class:

> *"A repository — 'repo' for short — is a project folder that Git is tracking the history of. We're creating this one on GitHub's website first, then pulling a copy down to your machine. From this point on, every assignment this semester lives inside this one repo, in its own dated or numbered folder."*

Do, live, in the browser: Click **New repository**. Name it exactly `ism2411`. Set visibility to **Public** (say why explicitly: a public repo is visible on your portfolio and to anyone with the link — appropriate for coursework, and worth contrasting briefly with *private* repos, which hide content entirely, appropriate for, say, a client's proprietary code). Check **Add a README**. Click **Create repository**.

Then, live, in the terminal:

```
git clone https://github.com/YOURUSERNAME/ism2411.git
cd ism2411
git status
```

Line-by-line explanation:

- `git clone https://github.com/YOURUSERNAME/ism2411.git` — downloads a full copy of the repo (including its entire history, even though there's only one commit so far — the automatic README-creation commit) to a new folder named `ism2411` in your current location. Point out explicitly: this single command does two things at once — creates the local folder *and* sets up the connection back to GitHub (called a **remote**) that `git push` will use later.
- `cd ism2411` — Module 02's navigation skill, moving into the newly cloned folder.
- `git status` — reports on the current state of the repo: which files are tracked, changed, staged, etc. Right after a clean clone, this should report "nothing to commit, working tree clean" — a good baseline to recognize, since students will run `git status` constantly from here forward to orient themselves.

Common student mistakes to watch for:

- Running `git clone` from inside an already-existing `ism2411` folder from an earlier module (Module 01's project folder!) — this creates confusing nested folders (`ism2411/ism2411/`) rather than an error. Have the room run `pwd` before cloning to confirm they're somewhere sensible — perhaps a dedicated `~/repos/` or `~/GitHub/` location, distinct from the informal `ism2411/` project folder from Module 01, and worth explicitly resolving that naming collision with the room up front.
- Typing the clone URL with their own placeholder text still in it (`YOURUSERNAME` literally, not

replaced) — produces a `fatal: repository not found` error; a good, low-stakes error to read together if it happens.
- Not being logged into GitHub in the browser when clicking “New repository” (landing on a sign-in page instead) — usually resolves itself, but worth a quick visual double-check.

Check for understanding: “What’s the difference between the repo that now exists on github.com and the folder that now exists on your own machine?” (They’re two separate copies of the same content, connected by the “remote” link `git clone` set up — changes made locally don’t appear on GitHub until explicitly pushed, and this two-copies-in-sync mental model is the foundation for everything else today.)

6.4 Exercise 3 — Add a `.gitignore` (0:25–0:35, 10 min)

Teaching goal: Understand what a `.gitignore` file does and why — and get the full stage → commit → push cycle under students' fingers for the very first time, on the lowest-stakes possible file.

Say to the class:

> *"Before we add any real code, one housekeeping file: `.gitignore`. This tells Git which files to* never *track — things that are personal to your machine, or auto-generated junk that shouldn't be part of the project's real history."*

Do: Create `.gitignore` in the repo root (in VS Code or any editor), with this content:

```
# Python
__pycache__/
*.pyc
*.pyo
.env
*.egg-info/

# OS
.DS_Store
Thumbs.db

# Editors
.vscode/
.idea/
```

Line-by-line explanation of the categories, not every individual line:

- **Python section** (`__pycache__/`, `*.pyc`) — Python automatically generates compiled bytecode files as a performance optimization when you run scripts; these are regenerated automatically on any machine and are pure clutter in a shared repo — never anything a human wrote or needs to see in the project's history.
- **OS section** (`.DS_Store` on Mac, `Thumbs.db` on Windows) — hidden files the operating system itself creates to remember folder view settings, icon positions, etc. — entirely irrelevant to the project's actual content, and different between every user's machine, so tracking them would create constant, meaningless diffs.
- **Editors section** (`.vscode/`, `.idea/`) — personal editor configuration (which files were open, window layout) — useful to *you*, on *your* machine, but not something a classmate or grader needs, and potentially different per person even on the identical project.
- **The general principle to state explicitly:** `.gitignore` doesn't delete these files or stop them from existing on your machine — it just tells Git "don't track changes to these," so they never get staged, committed, or pushed, keeping the repo's real history focused on files that

actually matter.

Now the stage → commit → push cycle, for real, for the first time:

```
git add .gitignore
git commit -m "add .gitignore for Python and OS files"
git push
```

Line-by-line explanation — this is the cycle every remaining exercise today reuses, so go slowly here:

- `git add .gitignore` — **stages** the file: tells Git "include this file's current state in the *next* commit I make." Staging is a genuinely separate step from committing — say explicitly: you can stage several files across several `git add` commands before committing them all together as one coherent unit, which is exactly what a good commit message describes.
- `git commit -m "add .gitignore for Python and OS files"` — takes everything currently staged and permanently records it as a new point in the repo's history, labeled with the message after `-m`. This is a **local** operation — at this point, the commit exists only on the student's own machine, not yet on GitHub.
- `git push` — uploads any local commits that GitHub doesn't have yet, up to the remote repo. This is the step that actually makes the change visible on github.com. A student who forgets this step will have a perfectly good local commit that a grader looking at their GitHub page will never see — worth stating as a real, common submission failure mode to watch for all semester.

Verify together: visit the repo on github.com and confirm the `.gitignore` file and its commit message are visible.

Common student mistakes to watch for:

- Running `git commit` without `-m` (or without a message at all) — this opens a text editor (often Vim, unfamiliar and genuinely confusing to a beginner) waiting for commit-message input. If this happens, help the student either type a message and save-and-quit (`:wq` in Vim specifically, worth writing on the board as an emergency escape hatch) or `Ctrl+C` out and re-run with `-m "..."` properly included.
- Forgetting `git push` entirely and assuming the commit alone finished the job — the most consequential mistake in this whole exercise, since it silently leaves work invisible on GitHub with no error at all. Make "did you push?" your default first question for the rest of the semester whenever a student says their GitHub doesn't show something they expect.
- Authentication failure on `git push` (password rejected) — GitHub requires a personal access token or SSH key, not a plain account password, over HTTPS. Resolve this using whichever single method you standardized on in your setup checklist, rather than improvising a second method mid-class for one student.

Check for understanding: "What's the difference between `git commit` and `git push` — could

I do one without the other, and what would that look like?" (`commit` records history locally; `push` uploads it. Yes, you could commit many times locally and push them all together later in one `git push` — worth demonstrating conceptually, even without doing it live here, since Exercise 6 will have students make exactly three separate commits before a final check.)

6.5 Exercise 4 — Add Module 7 Work (0:35–0:43, 8 min)

Teaching goal: Add a real, previously-written file (Module 07's `functions.py`) into an organized subfolder, and repeat the stage → commit → push cycle on genuinely meaningful content this time.

Say to the class:

> *"Same three-step cycle as Exercise 3, but now with actual coursework — your functions from last module, organized into a `week07/` folder. This folder-per-module structure is what your README will describe in a minute, and it's what every remaining module's submission will follow."*

Do, live:

```
mkdir week07
cp /path/to/your/functions.py week07/

git status
git add week07/
git commit -m "add module 7 functions: calculate_tax, apply_discount, final_price"
git push
```

Line-by-line explanation:

- `mkdir week07` — a new subfolder inside the repo, matching the module number — say explicitly that this naming convention (`week07`, not `Week 7` or `functions_folder`) matters for consistency across the whole semester's worth of submissions, and for the README you're about to write in Exercise 5.
- `cp /path/to/your/functions.py week07/` — copies the actual file from wherever it currently lives (likely inside last module's separate project folder) into this new location. Have students substitute their own actual path here — this is the one command in this lab that's genuinely different for every student, worth double-checking as you circulate.
- `git status` — run **before** `git add`, deliberately, to see the new `week07/` folder listed as "untracked" — this is worth pausing on: Git doesn't automatically track new files just because they exist in the folder; you have to explicitly `add` them. Contrast this with `.gitignore`'d files, which `git status` won't even mention.
- `git add week07/` — stages the *entire folder* at once (everything inside it), rather than naming the file individually — a useful shorthand worth calling out, since folders will often contain multiple files as the semester progresses.
- The commit message names the specific functions added, not just "add week 7" — model this explicitly as the standard to hold students to for the rest of the semester: a commit message should tell a reader, without opening the file, roughly *what changed*.

Verify together: visit `github.com/YOURUSERNAME/ism2411` and confirm the `week07/` folder and this commit message are both visible, per the lab page's own verification note.

Common student mistakes to watch for:

- Copying the file but forgetting to `cd` back into the `ism2411` repo folder first, accidentally creating `week07/` somewhere else entirely — a `pwd` check before starting, same habit from Module 02, resolves this quickly.
- Running `git add .` (the whole current directory) instead of `git add week07/` — not wrong, exactly, in this specific case (there's nothing else unstaged to worry about accidentally including), but worth flagging as a habit to be careful with: `git add .` stages *everything* in the current directory, including files a student might not have meant to include yet, which becomes a real risk once a repo has more going on in it.

Check for understanding: "If I run `git status` right now, immediately after the `git push` completed successfully, what should it report?" ("nothing to commit, working tree clean" again — same baseline as right after the clone, confirming everything local is now fully reflected on GitHub, with nothing pending.)

6.6 Exercise 5 — Write a README (0:43–0:51, 8 min)

Teaching goal: Markdown syntax (headings, bold, bullet lists) in a genuinely functional context — a README is the first thing anyone (a grader, a future employer, a collaborator) sees when visiting a repo, and GitHub renders it automatically on the repo's front page.

Say to the class:

> *"The README is your repo's front door — GitHub automatically displays it, formatted, right on the main page. We're writing it in Markdown, a lightweight formatting language you've actually already seen — every lab guide and reading in this course is written in it."*

Do, live, editing `README.md`:

```
# ISM2411 — Python for Business

**Name:** Your Name Here
**Semester:** Fall 2025
**Instructor:** [Course Instructor]

## About This Repo

This repository contains my weekly lab submissions for ISM2411.
Each folder corresponds to one module.

## Modules

- `week05/` — Conditionals: tiered discount calculator
- `week06/` — Loops: sales report with sum, average, max
- `week07/` — Functions: calculate_tax, apply_discount
- `week08/` — Git: first GitHub submission

## How to Run

Each script is standalone. Open a terminal and run:
\```
python week07/functions.py
\```
```

Line-by-line explanation of the Markdown syntax:

- `# ISM2411 — Python for Business` — a single `#` at the start of a line makes a top-level heading (renders large and bold on GitHub).
- `## About This Repo`, `## Modules`, `## How to Run` — `##` (two hashes) makes a second-

level heading, smaller than the title, used for each major section — say explicitly: more # characters means a *smaller* heading, which surprises some students expecting the opposite.
- **Name:** — double asterisks around text make it **bold** when rendered; the colon and following text stay normal weight since only the asterisk-wrapped portion is affected.
- – \week05/‘ — Conditionals: …– a leading- (dash, space) makes a bullet list item; the backticks aroundweek05/‘ render it in a monospace code font, a nice visual convention for filenames and code within otherwise-prose text.
- The triple-backtick fenced block for the “How to Run” command — same syntax this course’s own lab guides use for code blocks; renders as a distinct, monospaced, often syntax-highlighted block on GitHub.

Commit and push it, reusing the exact cycle from Exercise 3–4:

```
git add README.md
git commit –m "add README with module listing and run instructions"
git push
```

Common student mistakes to watch for:

- Forgetting the blank line between a heading and the paragraph below it, or between list items and surrounding text — Markdown is sometimes forgiving about this, sometimes not, and inconsistent spacing is the most common cause of a README that “looks wrong” on GitHub without any error message at all. If a student’s rendering looks off, have them view the *raw* file (a toggle GitHub provides) side by side with the rendered version to spot the issue visually.
- Using single asterisks (*bold*) expecting bold — single asterisks render as *italic*, not bold; double asterisks (**bold**) are required for bold. A quick, easy mix-up worth naming explicitly.

Check for understanding: “If I add a fifth module folder next week, what exactly do I need to update in this README to keep it accurate?” (Add one more bullet line under ## Modules — get a student to notice this file needs ongoing maintenance as the semester progresses, not just a one-time setup; this directly previews Exercise 6/Stretch 1’s back-filling work.)

6.7 Exercise 6 — Three Commits (0:51–0:59, 8 min)

Teaching goal: Confirm the whole session’s work by reading git log –-oneline — and reinforce that “three separate, coherent commits” is a deliberate discipline, not an accident of how the exercises happened to be structured.

Say to the class:

> *"You've actually already made three separate commits today, without necessarily thinking of it as a deliberate count — the `.gitignore`, the Module 7 folder, and the README. Let's confirm that with one command, and talk about why 'three separate commits' is better than 'one giant commit for everything.'"*

Live-code this:

```
git log --oneline
```

Line-by-line explanation:

- `git log` — shows the repo's full commit history, most recent first.
- `--oneline` — condenses each commit to a single line (a short hash plus the message), instead of the full multi-line default view (which also shows author, date, and full message) — a much more scannable format for a quick check like this.

Expected output shape (exact commit hashes will differ per student — that's expected and fine, hashes are unique to each commit's content and timing):

```
a3f1b2c add module 7 functions: calculate_tax, apply_discount, final_price
9d4e1a0 add .gitignore for Python and OS files
f7c2b3e add README with module listing and run instructions
```

Say explicitly, since this is the exercise's real point: "Notice these are three separate, single-purpose commits, each describing one coherent change — not one commit called 'update stuff' covering all three at once. If you were hiring someone and looked at their commit history, which would tell you more about how carefully they work?" This is a direct rehearsal of tonight's first reflection question — ask it now, out loud, before students see it as homework.

Verify on github.com too: visit the repo's commit history page (usually a "commits" link near the top of the repo) and confirm all three appear there as well, matching the local `git log` output — reinforcing the "local and remote should match after a successful push" mental model from Exercise 2.

Common student mistakes to watch for:

- Having fewer than three distinct commits because two exercises got accidentally combined into one `git add`/`git commit` cycle — not a disaster, but a good moment to ask the student to articulate, after the fact, what the "ideal" commit boundaries would have been, since recognizing good commit granularity is a skill independent of whether it was followed perfectly today.
- Confusing the *order* `git log` displays commits in — most recent commit at the **top** — with the order they were made in. If a student expects chronological top-to-bottom (oldest first), clarify explicitly that `git log`'s default and near-universal convention is newest-first.

Check for understanding: "The default branch on GitHub is often called `main` these days, though older repos or certain local Git configurations may default to `master` — does this distinction affect

anything you did in this lab?" (Not for this exercise specifically, but it's worth surfacing as an example of a small inconsistency students may encounter and should recognize rather than be thrown by — different repos, different defaults, same underlying Git concepts.)

6.8 Stretch 1 — Back-fill Earlier Modules (0:59–1:07, 8 min)

Teaching goal: Repeat the full add-a-module cycle independently, twice, without step-by-step guidance — the real test of whether the workflow actually transferred, not just whether students could follow along live.

Say to the class:

> *"Now do Exercise 4's process again, on your own, twice — once for Module 5's `discount.py`, once for Module 6's `sales_loop.py`. Separate commits for each, not one combined commit for both. Then update the README's module list."*

Facilitation notes rather than a live-coded demo — this is intentionally independent practice:

- Circulate and check specifically for **commit granularity** — a student who does `git add week05/ week06/` and commits both in one shot has technically completed the task but missed the actual point (practicing the discipline of separate, coherent commits). Redirect gently: "can you undo that and do it as two separate commits instead?" is a fine, low-stakes correction at this stage.
- The README update is easy to forget entirely, since it's a small, easy-to-overlook step after the "real" work of adding two folders — this is worth an explicit reminder mid-exercise, since a README that lists Modules 7–8 but not 5–6 is a subtle but real inconsistency a careful grader (or a future employer) would notice.

Verify: `git log --oneline` should now show five total commits (the original three plus two more), and the README should list all modules covered so far.

Common student mistakes to watch for:

- Copying `discount.py`/`sales_loop.py` from their original Module 05/06 project folders but accidentally also copying unrelated files alongside them (like an old `__pycache__/` folder) — a good live check that `.gitignore` from Exercise 3 is actually doing its job: those files should *not* show up as trackable in `git status`, even if physically present in the folder, confirming the ignore rules work as intended.

6.9 Stretch 2 Preview — Simulate a Bug and Rollback (as time allows)

Frame as a quick demo if time is short, since it's genuinely one of the more valuable "why Git matters" moments in the whole lab:

```
# Deliberately break something
git add functions.py
git commit -m "introduce bug for rollback exercise"
git push

# Then restore the working version
```

```
git revert HEAD
git push
```

One sentence of framing, said out loud, is worth more than a full live demo here if time is tight: “This is the entire reason version control exists: your mistake is never actually lost, and neither is the fix. `git revert` creates a *new* commit that undoes a previous one — the broken version stays visible in history (which is honest and often useful later), but the working code is restored. Confirm with `git log --oneline` afterward: you’ll see *both* the ‘introduce bug’ commit and a new ‘Revert …’ commit, not a history with the bug simply erased.” If you demo it live, verify with the class that the file’s content is genuinely restored to working order by re-running the script, not just trusting the command succeeded silently.

7 Wrap-Up (last ~8 minutes)

Review the reflection questions out loud:

1. *`git log --oneline` output, and what it would tell a hiring manager* — this was already rehearsed during Exercise 6's live discussion; use this moment to have a student restate it in their own words for the reflection.
2. *How Git changes your approach to editing a working script* — a strong answer names something like: "I'm less afraid to try something risky now, because I know I can always get back to the last commit" — the psychological safety of version control is a real, substantive point, not just a process one.
3. *One machine/workflow-specific `.gitignore` addition* — encourage something genuinely personal to their setup (a specific editor's config folder, a virtual environment folder like `venv/`, a personal notes file they keep in the project folder) rather than just repeating an entry already in the starter template.

Review the submission checklist together:

- ☐ Repo is named `ism2411`, set to Public
- ☐ `.gitignore` committed and pushed
- ☐ `week07/` folder with `functions.py` committed and pushed
- ☐ `README.md` with title, name, semester, description, and module list committed and pushed
- ☐ At least three separate, descriptively-messaged commits visible in `git log --oneline` and on github.com
- ☐ Repo URL submitted to Canvas

Preview Module 09: "Module 9 is the midterm — no new lab content, but the Git workflow you built today is exactly what you'll use to submit it, and every remaining assignment for the rest of the semester."

8 Appendix A — Full Command Reference

Exercise 2 (clone):

```
git clone https://github.com/YOURUSERNAME/ism2411.git
cd ism2411
git status
```

Exercise 3 (`.gitignore`, first commit cycle):

```
git add .gitignore
git commit -m "add .gitignore for Python and OS files"
git push
```

Exercise 4 (Module 7 work):

```
mkdir week07
cp /path/to/your/functions.py week07/
git status
git add week07/
git commit -m "add module 7 functions: calculate_tax, apply_discount, final_price"
git push
```

Exercise 5 (README):

```
git add README.md
git commit -m "add README with module listing and run instructions"
git push
```

Exercise 6 (verify history):

```
git log --oneline
```

Stretch 1 (back-fill, repeated per module):

```
mkdir week05
cp /path/to/your/discount.py week05/
git add week05/
git commit -m "add module 5 discount tier calculator"
git push

mkdir week06
cp /path/to/your/sales_loop.py week06/
git add week06/
git commit -m "add module 6 sales loop: sum, average, max"
git push
```

Stretch 2 (simulate bug, then roll back):

```
git add functions.py
git commit -m "introduce bug for rollback exercise"
git push

git revert HEAD
git push
```

Verified with a scratch repo: `git revert HEAD` (with no other flags) opens an editor for the revert commit's message — running `git revert --no-edit HEAD` skips that prompt and accepts Git's auto-generated "Revert '…'" message, which is a fine option to mention if a student gets stuck in an unfamiliar editor prompt (the same Vim-escape guidance from Exercise 3 applies here too).

9 Appendix B — Extra Practice (only if the class finishes early)

Six required exercises plus Stretch 1 fill the full 75 minutes at a normal pace, including realistic account-setup and authentication friction. If a section moves unusually fast:

Extra — a fourth, deliberately small commit. Have students add one more bullet point to the README (e.g., a "Contact" or "License" section) and commit it on its own, with its own descriptive message — good extra rehearsal of the full cycle on the smallest possible unit of change, reinforcing that a "commit" doesn't need to represent a large amount of work, just one coherent one.

Extra — inspect a single commit's changes. Have students run `git show` (with no arguments, showing the most recent commit) and read the diff output — lines starting with `+` were added, lines starting with `-` were removed. This previews a genuinely useful command for the rest of the semester: confirming *exactly* what changed in any given commit, not just that a change happened.